

Joe Thompson

Dr Huang

CS535

Southern Illinois University

10, December 2025

House Plant Disease Identifier

Introduction

Since moving out of my parents' home, it's been important to me to make any place I end up moving into more "homey." Over the years, I am a senior, I have bought, cared for, and subsequently watched die too many plants. Some went from root rot, others, a particularly cool tea bonsai tree, went out from mealy bugs. Since those mistakes, I have learned what to look out for and what to do when I see the symptoms, but not everyone has the time and patience to learn these distinctions. Everyone deserves to easily keep these plants. This is why I chose this project for my semester project. Additionally, I just think convolutional neural networks are just plain cool, but that was just a bonus. In this paper, I plan to describe the current research around plant disease identification, my niche in this research, how I initially planned to go about putting this niche into action, all the details on my dataset, the training process, and finally, my findings.

Research

Plant disease identification is no new concept in the world of convolutional neural networks (CNNs). When I started researching for this project, I found no shortage of research papers about plant diseases, how important it is to develop this technology, and so on. They all had one distinct caveat, though: vegetables and crops. I'll admit, that's definitely a more noble use of convolutional neural network technology, figuring out what's wrong with potatoes, tomatoes, and other crops so we can protect yields and help with food security. The paper I ended up leaning on the most for picking my models was *"Classification of Various Plant Leaf Disease Using Pretrained Convolutional Neural Network on ImageNet"* by Hukkeri et al. They use pretrained models like ResNet50, MobileNetV2, and EfficientNet on the PlantVillage dataset to classify different crop leaf diseases with really high accuracy. That's great for farmers, but PlantVillage is basically all fruits, vegetables, and field crops, not houseplants. It would have been super convenient if I could've just grabbed an already made dataset like that for my project, but I couldn't find anything that. So, I ended up having to make my own dataset from scratch. But at least I now know which model I should use

The Plan

The initial plan was to first create my own dataset of plants and diseases using a webcrawler to scrape for images (See below for the list), then train 3 CNN transfer learning models on that dataset (ResNet50, EfficientNetV2B0, and MobileNetv2), find which one was the best, and then maybe eventually make a web app for the best model. This plan, as it turns out, was not the best of plans, and I was wracked by one problem after another.

Plants: Pothos, Spider Plant, Snake Plant, Orchid, Cathedral Cactus, String of Turtles

Diseases: Spider Mites, Root Rot, Scale Insects, Mealy Bugs, Leaf Spot, Edema

The Plan: The Web Crawler

According to Cloudflare, a web crawler is “a software program that accesses, downloads, and/or indexes content from all over the Internet.” That’s exactly what my code did. For each plant, my code would first create a long list of search queries by combining the condition with general modifiers such as “indoor house plant photo” or “top-down view” or condition-specific modifiers like for root rot “root and soil visible” or “mushy roots close-up” with a list of negatives “-AI”, “-tattoo”, or “-plushie”. It then chose 10 of these search terms per condition, searched the web for a maximum of 100 images per search term. Ideally, this would leave me with 7000 images per plant that I would then need to clean. This may seem excessive, but this is before I clean the data; a lot of those images wouldn’t end up making it into the dataset. Additionally, the code would save the condition, the search term, and the folder it would go into, inside of a CSV file.

```
#from icrawler.builtin import GoogleImageCrawler
from icrawler.builtin import BingImageCrawler
import os, random
#turn off negatives

base_path = '/content/drive/MyDrive/MLPlant'

condition_list = ['pothos healthy',
                  'pothos root rot',
                  'pothos leaf spot',
                  'pothos mealybugs',
                  'pothos scale insects',
                  'pothos spider mites',
                  'pothos edema',]

negative_terms = [
    "art", "illustration", "AI", "generated", "wallpaper",
    "vector", "sticker", "tattoo", "poster", "toy", "decoration"
]
negative_suffix = " " + " ".join(f"-{t}" for t in negative_terms)

metadata = [] #store meta data in here add to csv at end

#Main loop: goes through the list of conditions
for i in condition_list:
    search_terms = [] #resets collected search terms per condition

    condition_folder = os.path.join(base_path, i.replace(" ", "_")) #makes new folder name for each condition inside of file path
    os.makedirs(condition_folder, exist_ok=True) # makes the actual folder
    special_modifiers = condition_modifiers.get(i) # queries python dictionary of special modifiers

    if special_modifiers is not None: #makes sure query didnt fail or is empty
        for k in special_modifiers: #loops through the dictionary entry adding the list of negaive suffixes
            #search_term = i + " " + k
            #search_terms.append(search_term)
            search_terms.append(i + " " + k + negative_suffix) #the condition + special modifier + negative search terms
            #end up with something like pothos root rot + root ball close up + the negatives

    for j in general_modifiers: #loops through general modifiers
        #search_term = i + " " + j
        #print(search_term)
        #search_terms.append(search_term)
        search_terms.append(i + " " + j + negative_suffix) # adds condition + general modifier + negative search terms

    random.shuffle(search_terms)
    search_terms = search_terms[:10] #grab a random set of 10 search terms from search term list
    print(search_terms)
    print(len(search_terms))

    for search_term in search_terms: #loop through our 10 search terms
        print(search_term)
        crawler = BingImageCrawler(storage={'root_dir': condition_folder}) # create the instance of our crawler
        crawler.crawl(keyword=search_term, max_num=100) # tell it what to look for (the search term)and the max amount we want of it
        metadata.append({
            'condition': i,
            'search_term': search_term, #grab the condition the search term and were we stored it
            'folder': condition_folder,
        })

    #saves meta data into a csv file
    meta_df = pd.DataFrame(metadata)
    meta_csv_path = os.path.join(base_path, 'pothos_search_metadata.csv')
    meta_df.to_csv(meta_csv_path, index=False)
    print("Saved metadata to:", meta_csv_path)
```

Unfortunately, the webcrawler ended up not working out. Every single time I ran it, the output was the same 33 images saved over and over again in every folder. I spent weeks trying to figure out what could be wrong with it. I tried different browsers (In the picture, I use Bing), I tried different crawlers, I scoured my code looking for logical mistakes. Finally, I just had to move on, I didn't have enough time to continue messing with it. What I now believe my issue was was that the browser was realizing that my code was a bot and would just serve the exact same pool of 33 images regardless of search terms. My solution was one that I was dreading from the start, but over the course of those weeks became louder and louder in the back of my head. I needed to make my dataset by hand.

The Plan: Making the Dataset

I was now making the dataset by hand, and I had a couple of realizations. Firstly, these images that I was finding sucked. For the CNN to be as accurate as possible, good, clear images are paramount. Additionally, the images can't have any words, can't have more than one plant in them, and so on. The images I was finding quite commonly were anything but that. Especially for diseases. The vast majority of these images were either people asking what was wrong with their plant, or people comparing the differences between diseases. I was actually kind of glad I was doing it by hand. If my web crawler had ended up working out, I would have had to have gone through so many bad images. And since I was doing it by hand, I was able to go to the site where people were asking what was wrong with their plant and was able to determine what disease was afflicting it based off of what people in the comments of the form were saying. This is definitely not the most scientific way of getting solid labels but I was desperate. This leads to my second realization and what I ended up doing.

Secondly, making the data set by hand means I would have to scale down, by a lot. I decided that instead of six plants and seven conditions (six diseases+healthy), I would do one plant and four conditions. I would combine my disease into three different classes, as I was having a lot of trouble finding images of specific diseases on specific plants. What I ended up deciding upon was that I would only do pothos for my plant and my four conditions would be: healthy, root conditions (root rot, overwatering and edema; all caused by overwatering your plant), leaf spot (viral, bacterial, or nutrient deficiency), and pests (Mealy Bugs, Spider Mites, Aphids, Scale insects, Fungal Gnats). With these four classes I went about making my dataset.

Preprocessing: The Dataset

After too many long hours of searching and downloading images, I finally came out with 150 healthy pothos pictures, 120 leaf spots, 86 pests, and 101 root conditions. 457 is definitely a lot of images, but definitely not enough to go about doing this project. I ended up doing a number of things to the dataset. Firstly, I removed any accidentally repeated images, I added variance, resized the images to be 224x224, and finally randomly split the data into training, validation, and test.

Preprocessing: The Duplicates

To find the duplicates in the dataset, the function first loops through my main folder and grabs a folder. It loops through that folder, saves the files inside (the images) it loops through the folder, finds the hash map of the image and then checks to see if that hash is in the list of seen hashes. If it is, then it either deletes or it prints the file name if it's a dry run. I wanted to include the dry run aspect because I was hesitant to delete things if it didn't work the first time.

```

def remove_duplicates(base_path, folders, dry_run=True):

    # include .webp now
    valid_exts = ('.jpg', '.jpeg', '.png', '.webp')

    seen_hashes = {}    # hash -> first file path seen
    removed_files = []  # list of files that are (or would be) removed

    for folder in folders: # loop through list of folders
        folder_path = os.path.join(base_path, folder)
        files = [
            f for f in os.listdir(folder_path) #gets the files from the folders
            if f.lower().endswith(valid_exts)
        ]
        for fname in files: #loops through files per folder
            fpath = os.path.join(folder_path, fname)
            # read file and compute md5 hash
            with open(fpath, 'rb') as f: #opens file reads hashmap
                file_hash = hashlib.md5(f.read()).hexdigest()

            if file_hash in seen_hashes: #if the has is the same as one in seen hases
                # duplicate found
                if dry_run:
                    print(f"[DRY RUN] Would remove duplicate: {fpath}") #print if dry run
                else:
                    os.remove(fpath)
                    print(f"Removed duplicate: {fpath}") #delete if real
                    removed_files.append(fpath)
            else:
                # first time seeing this hash
                seen_hashes[file_hash] = fpath

    if dry_run:
        print(f"\n[DRY RUN] Found {len(removed_files)} duplicates. No files were deleted.")
    else:
        print(f"\nRemoved {len(removed_files)} duplicate files.")

    return removed_files

```

Preprocessing: Variance

My next step in preprocessing, after removing any duplicates, was to artificially increase the size and variability of the dataset through data augmentation. This would help me increase my number of images. I did several things to do this. At a high level, for each image, my code would create four copies of the image. For one of the images, it would randomly rotate it 20 degrees to the left or the right. For the next, it would randomly crop and zoom into the image 80%-100% of the original image. The next image it added plus or minus 20% contrast. Finally, it would add plus or minus 20% brightness. Doing all this essentially quadrupled my dataset while adding variation for training.

```

def augment_image(img):

    aug_images = []

    # flip
    aug_images.append(img.transpose(Image.FLIP_LEFT_RIGHT))

    #small random rotation -20 to +20 degrees
    angle = random.uniform(-20, 20)
    aug_images.append(img.rotate(angle, resample=Image.BICUBIC, expand=True))

    # random crop/zoom 80-100% of original then resize back
    w, h = img.size
    scale = random.uniform(0.8, 1.0) # 80-100% crop
    new_w, new_h = int(w * scale), int(h * scale)
    left = random.randint(0, w - new_w)
    top = random.randint(0, h - new_h)
    crop = img.crop((left, top, left + new_w, top + new_h))
    aug_images.append(crop.resize((w, h), Image.BICUBIC))

    # brightness/contrast jitter +20% -20%
    b_factor = random.uniform(0.8, 1.2)
    c_factor = random.uniform(0.8, 1.2)
    bright = ImageEnhance.Brightness(img).enhance(b_factor)
    contrast = ImageEnhance.Contrast(bright).enhance(c_factor)
    aug_images.append(contrast)

    return aug_images

for cls in classes:
    input_folder = os.path.join(base_path, cls)
    output_folder = os.path.join(output_root, cls)
    os.makedirs(output_folder, exist_ok=True)

    print(f"Processing class: {cls}")

    for fname in os.listdir(input_folder): #loops through file names
        if not fname.lower().endswith((".jpg", ".jpeg", ".png", ".webp")): #if file name isnt one of these it skips
            continue

        in_path = os.path.join(input_folder, fname)
        # make RGB
        img = Image.open(in_path).convert("RGB")

        base_name, ext = os.path.splitext(fname)

        #save og copy
        orig_out = os.path.join(output_folder, f"{base_name}_orig{ext}")
        img.save(orig_out)

        # save change versions
        for idx, aug_img in enumerate(augment_image(img), start=1):
            out_path = os.path.join(output_folder, f"{base_name}_aug{idx}{ext}")
            aug_img.save(out_path)

    print("Augmented dataset saved in:", output_root)

```

Preprocessing: Resizing

Next, I dealt with resizing and adding padding to the images. All the images needed to be 224x224 for training. First, I had a function that scales an image so that it fits inside 224x224 without distorting the aspect ratio, then it centers in on a black background to fill out the square. In the main loop, the code goes through each folder, reads every image, converts it to grayscale, blurs it, and uses thresholding and contours to find the largest object in the image, the pothos plant. The code then takes

the bounding box of that contour, skips the bad boxes, adds padding around the pothos, crops to that region, and then calls the function to resize the image. After all this, hopefully, you end up with a centered image that is 224x224.

```
# resize and add padding
def resize_and_pad(img, size=(224, 224)):
    h, w = img.shape[:2]
    scale = min(size[0] / h, size[1] / w)
    nh, nw = int(h * scale), int(w * scale)
    img_resized = cv2.resize(img, (nw, nh))

    # create new image and center inside
    result = np.zeros((size[0], size[1], 3), dtype=np.uint8)
    y_offset = (size[0] - nh) // 2
    x_offset = (size[1] - nw) // 2
    result[y_offset:y_offset+nh, x_offset:x_offset+nw] = img_resized

    return result

for plant_folder in plant_folders:
    input_folder = os.path.join(base_path, plant_folder)
    output_folder = os.path.join(base_output_path, plant_folder)
    os.makedirs(output_folder, exist_ok=True)

    print(f"Processing folder:{plant_folder}")

    for filename in os.listdir(input_folder):
        if not filename.lower().endswith(('.jpg', '.jpeg', '.png', '.webp')):
            continue # openCV cant handle avif files, just leave them alone they wont be in completed dataset

        path = os.path.join(input_folder, filename)
        img = cv2.imread(path)
        if img is None:
            print(f" Skipping unreadable file: {filename}")
            continue

        # Convert to grayscale blur and threshold
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        blurred = cv2.GaussianBlur(gray, (5, 5), 0)
        _, thresh = cv2.threshold(
            blurred, 50, 255,
            cv2.THRESH_BINARY + cv2.THRESH_OTSU
        )

        # find contours assume largest is main plant/leaf area
        contours, _ = cv2.findContours(
            thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
        )

        if not contours:
            print(f" No contours found, skipping: {filename}")
            continue

        best_contour = max(contours, key=cv2.contourArea)
        x, y, w, h = cv2.boundingRect(best_contour)

        if w < 50 or h < 50 or w/h > 4 or h/w > 4:
            print(f" Weird bounding box, skipping: {filename}")
            continue

        # Add padding around the crop
        pad = int(max(w, h) * 0.2)
        x1 = max(0, x - pad)
        y1 = max(0, y - pad)
        x2 = min(img.shape[1], x + w + pad)
        y2 = min(img.shape[0], y + h + pad)
        cropped = img[y1:y2, x1:x2]
        resized = resize_and_pad(cropped, size=(224, 224))

        out_path = os.path.join(output_folder, filename)
        cv2.imwrite(out_path, resized)

    print(f"Finished processing {plant_folder}")

print("Cropped dataset at:", base_output_path)
```

Preprocessing Splitting

Finally, my code takes the now augmented and cropped dataset and splits it randomly into training, validation, and test sets for each class. It starts with a single input folder that contains one subfolder per class (healthy leaf spot, pests and root rot/root condition). It then creates a new output directory where each class is organized under train/, val/, and test/. For each class it lists all image files, shuffles them with a fixed random seed, and then divides them into 70% training 15% validation, and 15% test. Finally, it copies the images into the corresponding split folders and prints how many images ended up in each set for each class.

```
# AUGMENTED + CROPPED224 dataset
input_folder = '/content/drive/MyDrive/MLPLANT3_aug_cropped224'

output_base = input_folder + "_split" # /content/drive/MyDrive/MLPLANT3_aug_cropped224_split

splits = ['train', 'val', 'test']

plant_folders = [
    "pothos_healthy",
    "pothos_leaf_spot",
    "pothos_pests",
    "pothos_root_rot",
]

# make train val and test folders
for split in splits:
    for plant in plant_folders:
        os.makedirs(os.path.join(output_base, split, plant), exist_ok=True)

# Split ratios
train_ratio = 0.7
val_ratio   = 0.15
test_ratio  = 0.15

# split seed
random.seed(42)

#split images for each class
for plant in plant_folders:
    species_path = os.path.join(input_folder, plant)
    images = [
        f for f in os.listdir(species_path)
        if f.lower().endswith(('.jpg', '.jpeg', '.png', '.webp'))
    ]

    random.shuffle(images)

    n_total = len(images)
    n_train = int(n_total * train_ratio)
    n_val   = int(n_total * val_ratio)

    train_files = images[:n_train]
    val_files   = images[n_train:n_train + n_val]
    test_files  = images[n_train + n_val:]

    #copy train files
    for img_file in train_files:
        shutil.copy(
            os.path.join(species_path, img_file),
            os.path.join(output_base, 'train', plant, img_file)
        )

    #copy val files
    for img_file in val_files:
        shutil.copy(
            os.path.join(species_path, img_file),
            os.path.join(output_base, 'val', plant, img_file)
        )

    # copy test files
    for img_file in test_files:
        shutil.copy(
            os.path.join(species_path, img_file),
            os.path.join(output_base, 'test', plant, img_file)
        )

    print(f"{plant}: {len(train_files)} train, {len(val_files)} val, {len(test_files)} test")

print("Finished splitting dataset into train/val/test")
print("Split dataset at:", output_base)
```

Preprocessing: Finished

With all this preprocessing completed, we are left with 2,112 images that have the duplicates removed, augmented to have variance resized and padded to be 224x224, and split into 70% training, 15% validation, and 15% test. There are 665 healthy images, 559 leaf spots, 412 pests, and 476 root conditions. Once shuffled and split, here are the numbers per set:

```
pothos_healthy: 465 train, 99 val, 101 test
pothos_leaf_spot: 391 train, 83 val, 85 test
pothos_pests: 288 train, 61 val, 63 test
pothos_root_rot: 333 train, 71 val, 72 test
Finished splitting dataset into train/val/test
Split dataset at: /content/drive/MyDrive/MLPLANT3_aug_cropped224_split
```

Model Trainings

The model training on this project was fairly smooth. To start, though, I think that I'll take a second to talk about what transfer learning is. Basically means taking CNNs that were already trained on a huge dataset and reusing them for a new task. In the paper *"Classification of Various Plant Leaf Disease Using Pretrained Convolutional Neural Network on ImageNet"* by Hukkeri et al. they classified PlantVillage leaf disease images. Instead of training everything from scratch, they keep most of the pre-trained model the same, remove some of the original layers, freeze the existing weights, and then bolt on new layers (activation, batch-normalization, and dense layers) that are trained specifically on the PlantVillage dataset. I did basically the same approach with my training. I first froze the model. Added my own initial layers and trained that model.

```
# freeze
resnet_base.trainable = False

x = resnet_base.output
x = GlobalAveragePooling2D()(x)      # pool spatial features
x = Dense(256, activation='relu')(x)  # small dense layer
x = Dropout(0.5)(x)                  # dropout for regularization
outputs = Dense(NUM_CLASSES, activation='softmax')(x)

resnet_model = Model(inputs=resnet_base.input, outputs=outputs)
```

After I trained the model, I unfroze a portion of it, and then retrained the unfrozen section for fine-tuning. I had one major hiccup while doing all this first with ResNet50, kept getting absolutely terrible accuracy metrics. Like 41.41% test accuracy. I added a lot in an attempt to fix this issue. ResNet was severely struggling to classify both pests and root conditions, so I added weighted classes to the model to potentially help the model classify them. They were my two smallest classes, so I thought maybe it could be due to that. That wasn't it. I tried adding early stopping to the model. I set the epochs to 50 and implemented early stop such that if the training accuracy didn't improve for 5 epochs, it would stop. That helped slightly, but it still wasn't it. I also tried it up such that it would reduce the learning rate if the accuracy plateaued for two epochs in a row. Finally, I found it. When I first initialized the images

before i fed them into the model, I had mistakenly set rescale = 1./255 rather than what ResNet50 expects rescale to be set to resnet's own preprocessing scale, resnet_preprocess. For the rest of the project, I just took what I learned from ResNet50 and applied it to both of them, changing little.

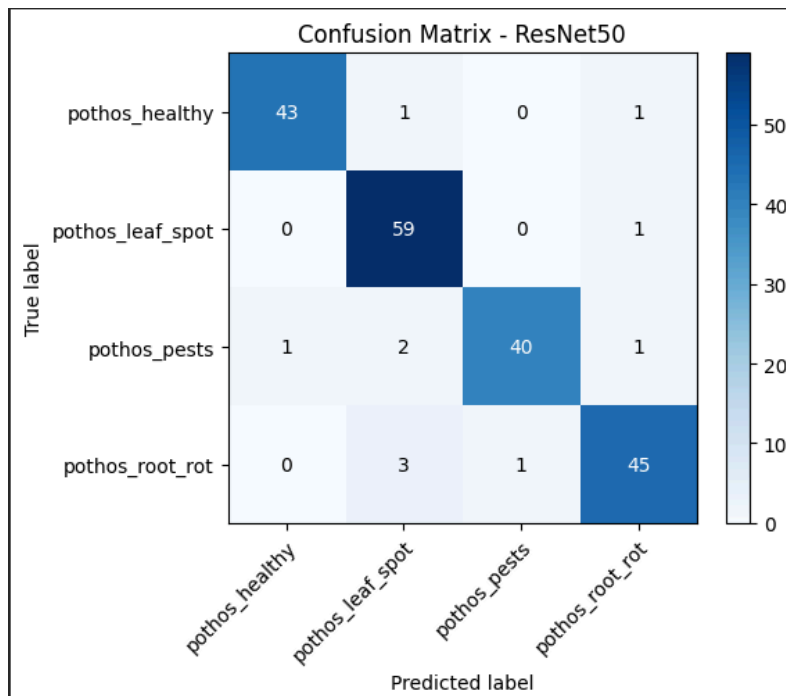
Metrics

After the training, the metrics were actually quite good considering the models were trained on a smaller dataset.

ResNet50

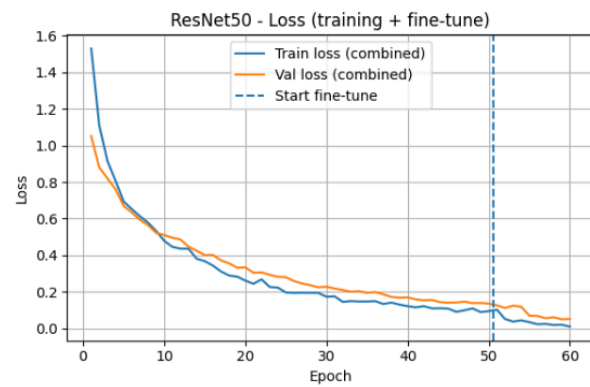
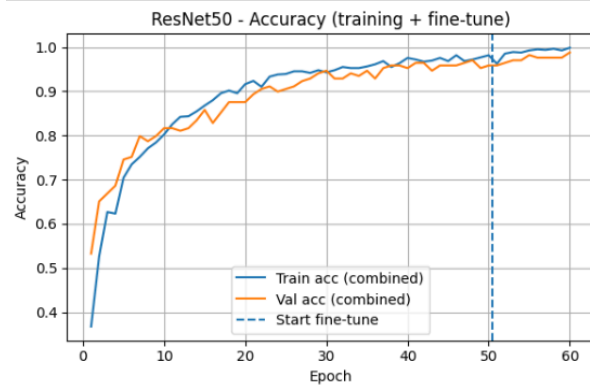
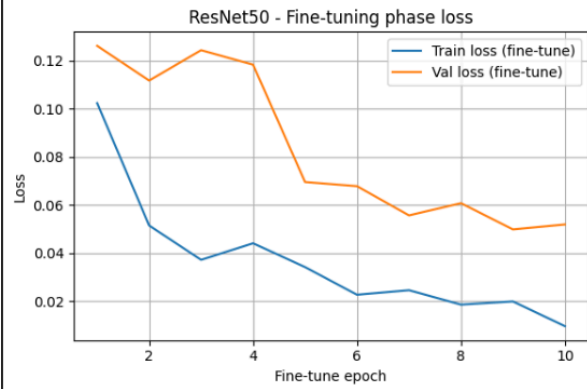
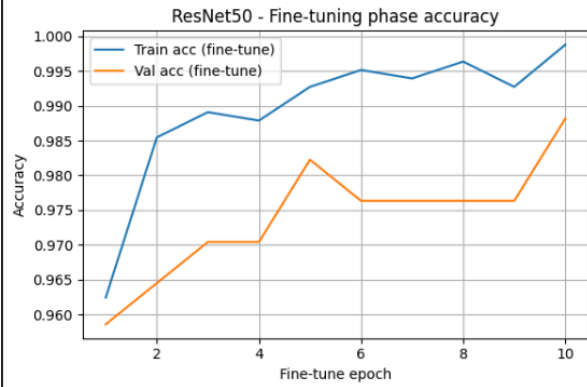
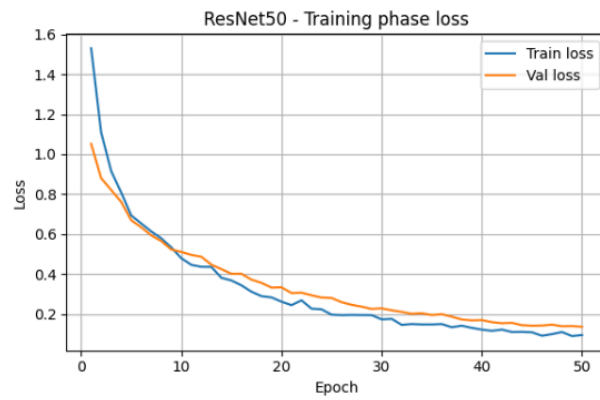
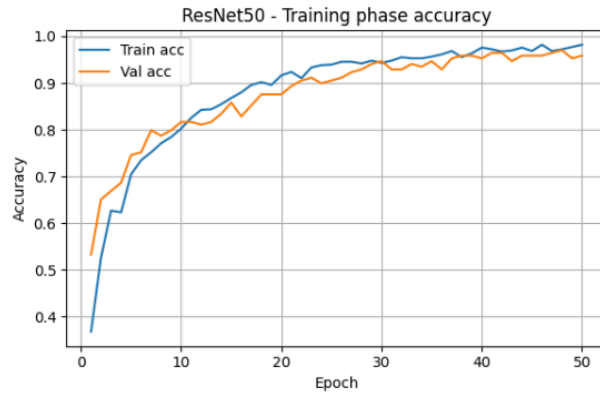
Test Accuracy: 94.44%

Test Loss: 0.0938



Classification report:

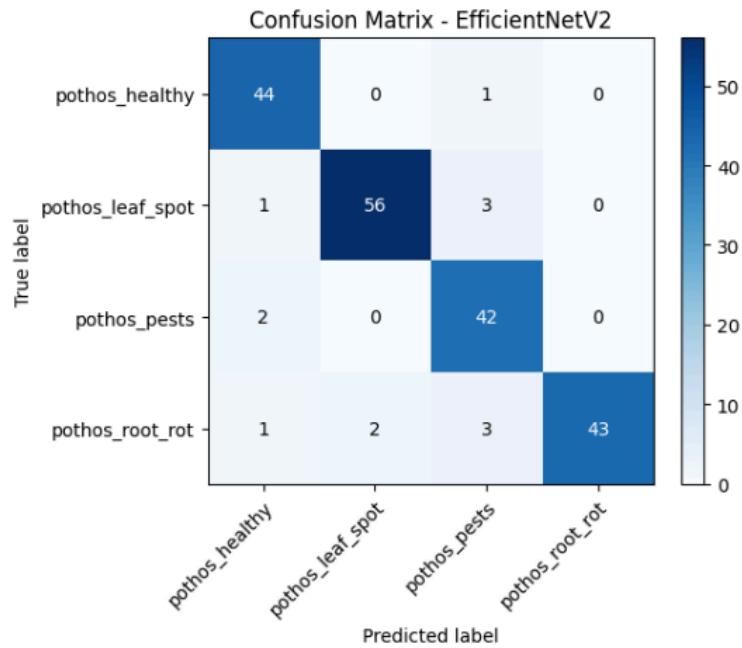
	precision	recall	f1-score	support
pothos_healthy	0.98	0.96	0.97	45
pothos_leaf_spot	0.91	0.98	0.94	60
pothos_pests	0.98	0.91	0.94	44
pothos_root_rot	0.94	0.92	0.93	49
accuracy			0.94	198
macro avg	0.95	0.94	0.94	198
weighted avg	0.95	0.94	0.94	198



EfficientNetV2B0

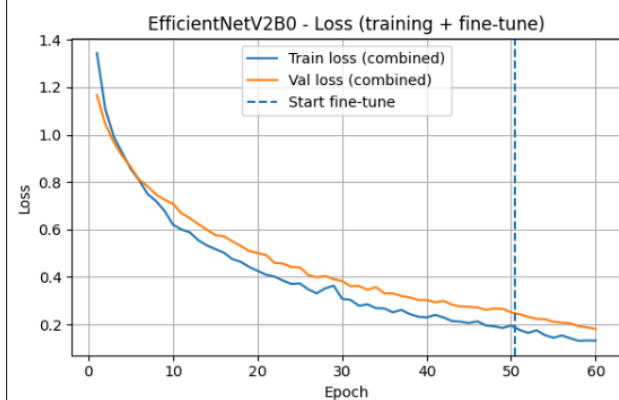
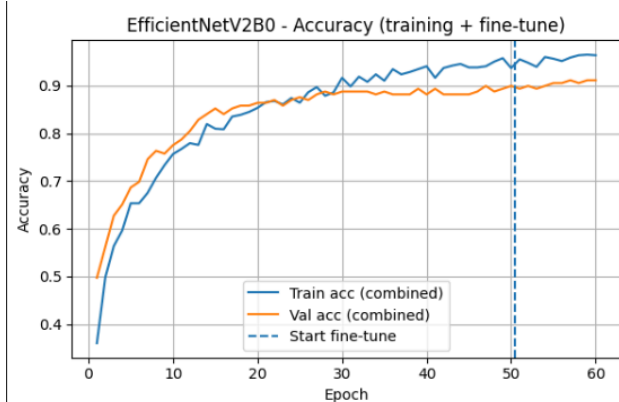
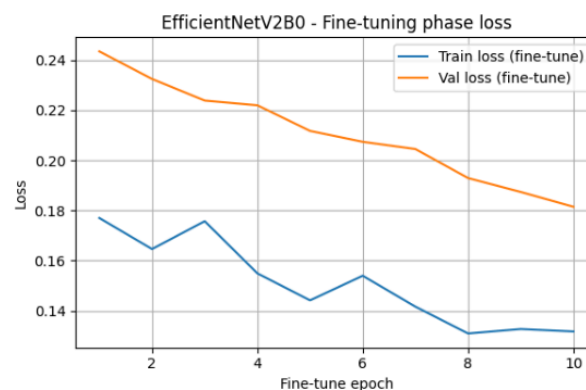
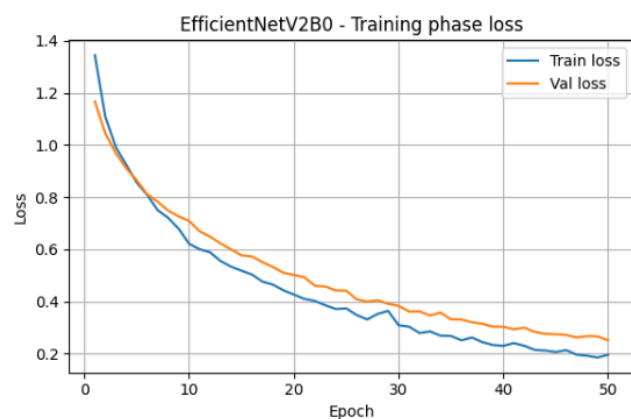
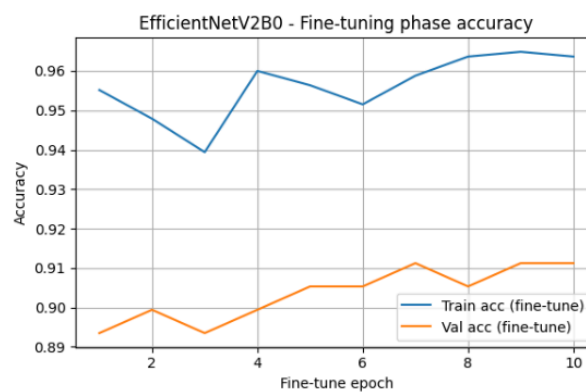
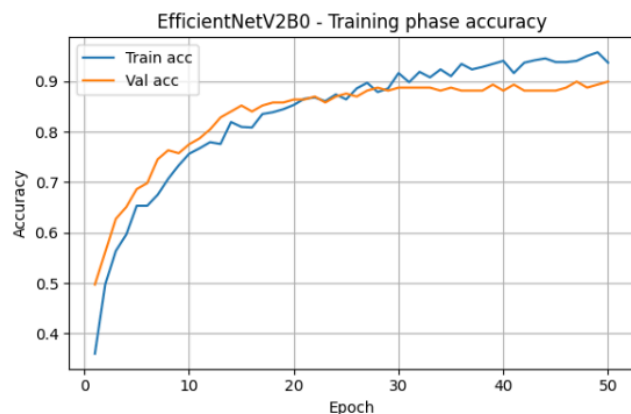
Test Accuracy: 93.43%

Test Loss: 0.2541



Classification report:

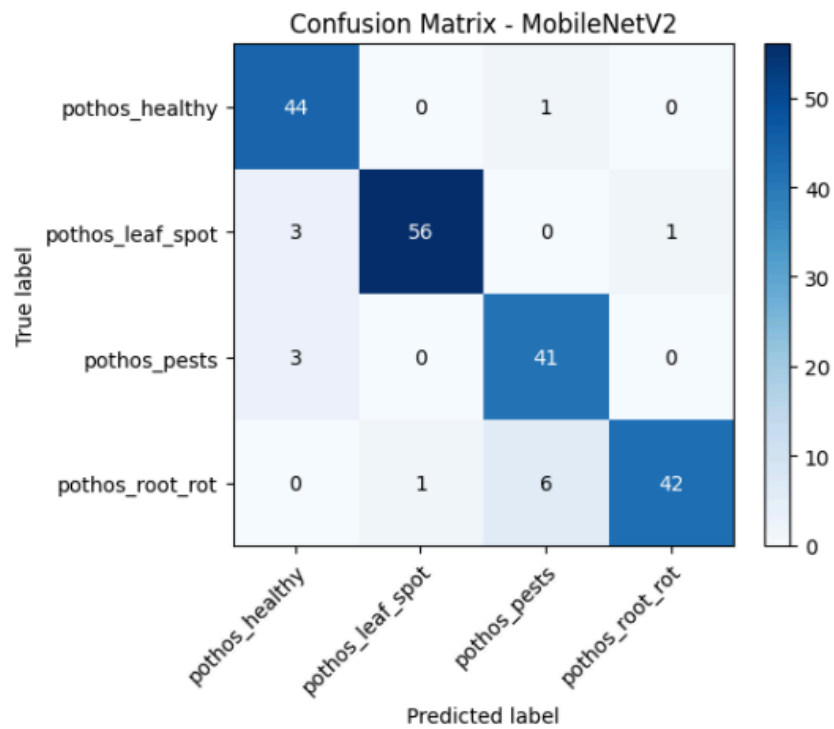
	precision	recall	f1-score	support
pothos_healthy	0.92	0.98	0.95	45
pothos_leaf_spot	0.97	0.93	0.95	60
pothos_pests	0.86	0.95	0.90	44
pothos_root_rot	1.00	0.88	0.93	49
accuracy			0.93	198
macro avg	0.93	0.94	0.93	198
weighted avg	0.94	0.93	0.93	198



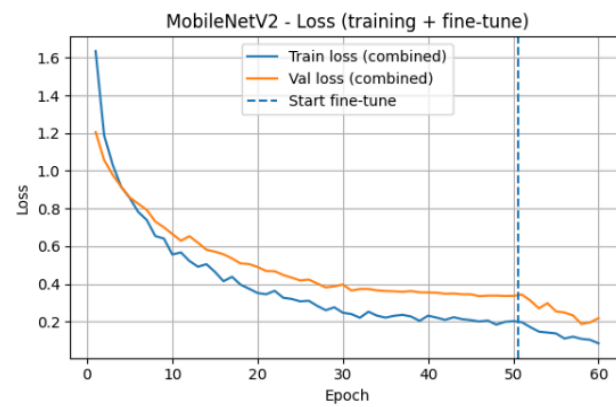
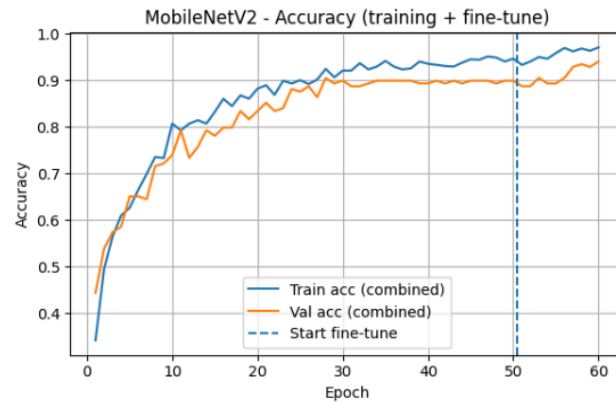
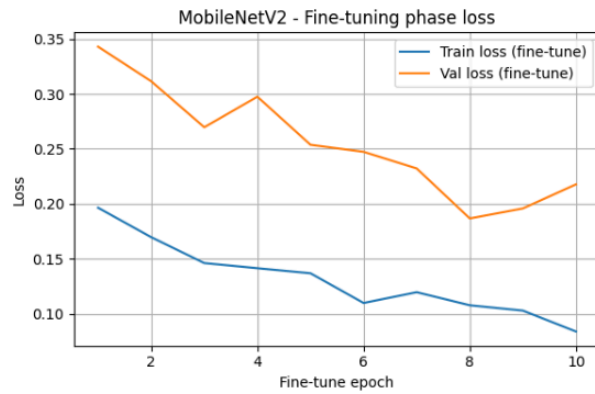
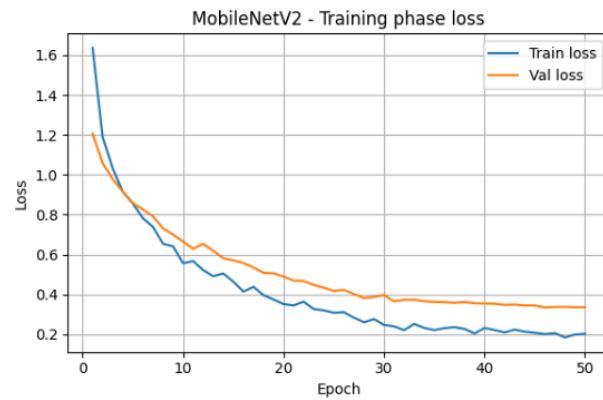
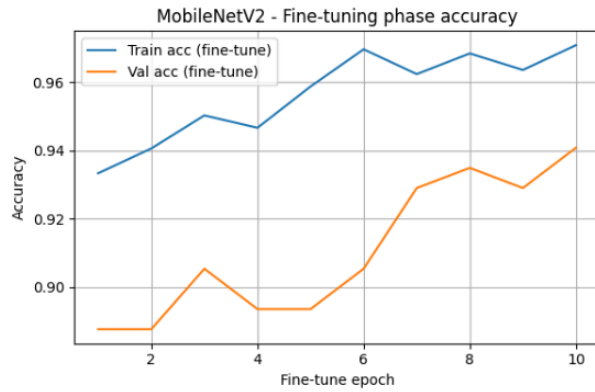
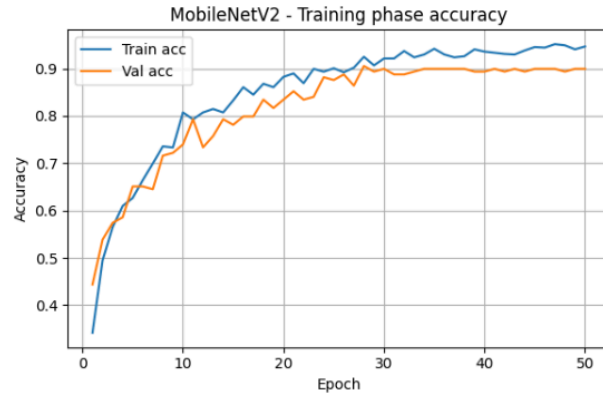
MobileNetV2

Test Accuracy: 92.42%

Test Loss: 0.2127



Classification report:				
	precision	recall	f1-score	support
pothos_healthy	0.88	0.98	0.93	45
pothos_leaf_spot	0.98	0.93	0.96	60
pothos_pests	0.85	0.93	0.89	44
pothos_root_rot	0.98	0.86	0.91	49
accuracy			0.92	198
macro avg	0.92	0.93	0.92	198
weighted avg	0.93	0.92	0.92	198



Metrics: Comparison

	Model	Test Accuracy	Test Loss	Macro Precision	Macro Recall	Macro F1	Weighted F1	Params (Millions)
0	ResNet50	0.9444	0.1137	0.9495	0.9416	0.9448	0.9444	24.11
1	EfficientNetV2B0	0.9343	0.2541	0.9348	0.9358	0.9333	0.9347	6.25
2	MobileNetV2	0.9242	0.2127	0.9233	0.9250	0.9220	0.9246	2.59

Looking at the metrics, all three models did really well, I expected them to do much worse like what it was when I first started working on it. ResNet50 ended up doing the best overall with 94.44% test accuracy, and its F1 score matches that at 0.9444; it does well across the board on all four classes. EfficientNetV2B0 also did quite well with 93.43% accuracy and an F1 score of 0.9333. MobileNetV2 did the worst but just barely. ResNet50 had the most parameters by far, though with 24 million, which is probably why it did the best overall.

Conclusion

Overall, I learned so much doing this project. Last spring, I did a similar project, an aquarium fish identifier, but with a partner. By doing it myself, I learned all the little things I missed the last time. Last time I mainly focused on the training of the models and the metrics of the model. This time I was able to go super in-depth into the data collection and data preprocessing, and I got to learn what exactly goes into creating, cleaning, and curating a dataset for a CNN. It was fascinating, and I was so glad I chose something that meant something to me. I think that it's imperative to choose topics you are truly interested in for these projects. It makes all the problems and setbacks seem just like bumps in the road. From my issues with my web crawler to my mistakes in my training, I felt like I truly learned a lot from this project, and I look forward to seeing how I can improve or move on. I do think that my data collection could have been done better, and that could definitely be a way for me to improve the accuracy of all three models. I did my best with identifying the diseases, but I am not a biologist, so there's definitely room for improvement in that area. Additionally, after adding variance with the data augmentation. My dataset is pretty small: 4 classes, 2,112 images, is nothing compared to PlantVillages' 14 classes, 54,303 images (PlantVillage is the dataset *"Classification of Various Plant Leaf Disease Using Pretrained Convolutional Neural Network on ImageNet"* used)

References

- “What Is a Web Crawler?” *Cloudflare*, n.d., <https://www.cloudflare.com/learning/bots/what-is-a-web-crawler/>. Accessed 9 Dec. 2025.
- Hukkeri, Geetabai S., et al. “Classification of Various Plant Leaf Disease Using Pretrained Convolutional Neural Network on ImageNet.” *The Open Agriculture Journal*, vol. 18, 2024, e18743315305194, <https://doi.org/10.2174/0118743315305194240408034912>
- Ferentinos, K. P. (2018). “Deep learning models for plant disease detection and diagnosis.” *Computers and Electronics in Agriculture*, 145, 311–318. <https://www.sciencedirect.com/science/article/pii/S0168169917311742#s0010>
- “tf.keras.applications.MobileNetV2.” *TensorFlow API Documentation*, TensorFlow, n.d., www.tensorflow.org/api_docs/python/tf/keras/applications/MobileNetV2
- tf.keras.applications.ResNet50.” *TensorFlow API Documentation*, TensorFlow, n.d., www.tensorflow.org/api_docs/python/tf/keras/applications/ResNet50
- “tf.keras.applications.EfficientNetV2B0.” *TensorFlow API Documentation*, TensorFlow, n.d., www.tensorflow.org/api_docs/python/tf/keras/applications/EfficientNetV2B0

Technologies Used

Data Collection

- Python (os, shutil) for organizing dataset folders
- hashlib to detect and remove duplicate images
- random (with seed) for consistent train/val/test split
- Pillow / PIL (Image, ImageEnhance) for resizing and augmenting images
- OpenCV (cv2) for leaf cropping, resizing, and background cleanup
- NumPy (numpy) for array manipulation while processing images
- Pandas (pandas) to store metadata into CSV
- Google Colab + Drive integration for dataset processing and storage

Training

- TensorFlow / Keras (core deep learning framework)
- Transfer learning models: ResNet50, EfficientNetV2B0, MobileNetV2
- Keras layers: Dense, Dropout, GlobalAveragePooling2D
- Keras preprocessing: ImageDataGenerator + preprocess_input
- Optimizer: Adam
- Callbacks: EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
- Class weighting with compute_class_weight (scikit-learn)
- Evaluation tools: confusion_matrix, classification_report (scikit-learn)

- Matplotlib for visualization (confusion matrix, plots)
- NumPy (numpy) for converting predictions and numerical operations